PROYECTO SEMESTRAL · HITO 2 · EQUIPO 05

MediAgenda

Integración del frontend con PHP: la maqueta cobra vida en el servidor



Benjamin Concepción · Dana Wing · Alvaro Guardia · Edwin Vergara

Noviembre de 2026

EQUIPO

Roles rotados: cada uno lideró una capa nueva

DW

Dana Wing

@dalilaling

Líder de proyecto y repositorio

Hito 1: calidad y W3C

AG

Alvaro Guardia

@aguard28

Diseño y experiencia de usuario

Hito 1: calidad y W3C

BC

Benjamin Concepción

@bconcepcion1376

Desarrollo (backend PHP)

Hito 1: líder y HTML/CSS

EV

Edwin Vergara

@simbiak

Calidad, seguridad y documentación

Hito 1: documentación

Sustentación por sorteo: cualquiera de los cuatro puede explicar cualquier parte del sistema.

DEL HITO 1 AL HITO 2

De una maqueta estática a un sistema que responde

HITO 1 · MAQUETA

Páginas .html con action="#"

Filas de tablas escritas a mano

Encabezado y pie copiados en cada página

HITO 2 · PHP 8.2 EN EL SERVIDOR

Los formularios se procesan y validan

Listados con foreach desde datos_demo.php

includes/ reutilizados con require_once

Sesión, roles y contraseñas cifradas

12

páginas .php

migradas desde la maqueta

6

scripts procesar_*

reciben los formularios

4

clases propias

Validador, Cita, Horario, Usuario

4

includes

funciones, encabezado, menú y pie

19

casos de prueba

repartidos entre el equipo

29

requerimientos

en la matriz de trazabilidad

ESTRUCTURA DEL PROYECTO

Cada página se arma con piezas reutilizables

```
proyecto-semestral-eq05/
├── index.php
├── principal_*.php
├── secundaria_formulario.php
├── procesar_*.php
├── clases/
│   ├── Validador.php
│   ├── Cita.php
│   ├── Horario.php
│   └── Usuario.php
├── config/datos_demo.php
├── includes/
│   ├── funciones.php
│   ├── encabezado.php
│   ├── menu.php
│   └── pie.php
├── css/   img/   docs/
```

ORDEN DE CARGA DE UNA PÁGINA

1. **`includes/funciones.php`**
   sesión, limpiar(), redirigir(), exigir_rol()
2. **`config/datos_demo.php`**
   médicos, pacientes, citas, feriados
3. **`includes/encabezado.php`**
   carga menu.php según el rol
4. **contenido de la página**
   foreach sobre los arreglos
5. **`includes/pie.php`**
   cierre común de todas las páginas

Archivos protegidos

La página que sí se abre define:

```
define('APP_ACCESO', true);
```

Cada include y clase comprueba:

```
defined('APP_ACCESO')
  or exit('Acceso denegado');
```

.htaccess con «Deny from all» en clases/, includes/ y config/: con Apache ni siquiera se sirven.

FLUJO DE UNA PETICIÓN

Qué pasa cuando el paciente pulsa «Confirmar cita»

NAVEGADOR

`principal_formulario.php`

El paciente completa la cita y envía el formulario con method="post".

POST

SERVIDOR

`procesar_principal.php`

1. **¿REQUEST_METHOD es POST?**
   si no, vuelve al formulario
2. **Lee cada campo con trim()**
   $_POST[...] ?? ''
3. **Valida con la clase Validador**
   campo por campo: requerido, formato
4. **Aplica las reglas de la agenda**
   24 h, feriados, horario, R1, R2, R3

Hay errores

Guarda errores y datos en $_SESSION y redirige. El formulario muestra los mensajes, conserva lo escrito y hace unset().

Todo es válido

Crea el objeto Cita y muestra la confirmación con los datos saneados con limpiar().

Cada formulario usa el método que le corresponde

GET

Consulta o filtra: no cambia datos y la URL se puede guardar

Buscador de disponibilidad (inicio)
`principal_formulario.php?especialidad=…`

Filtros del listado de citas
`principal_listado.php?estado=…`

Paginación del listado
`principal_listado.php?pagina=2`

Detalle de una cita
`principal_detalle.php?codigo=CIT-0201`

Horario de un médico
`secundaria_formulario.php?medico=rios`

POST

Envía datos personales o contraseñas, o modifica datos

Inicio de sesión
`procesar_login.php`

Registro de paciente
`procesar_registro.php`

Reservar cita
`procesar_principal.php`

Horario del médico
`procesar_secundaria.php`

Nota de consulta
`procesar_nota.php`

Regla práctica: si la URL se puede compartir sin riesgo, GET; si lleva una clave o cambia algo, POST.

Cuatro clases con propiedades privadas

Validador

Reúne los errores de un formulario

`private $errores`

requerido(), longitud(), correo()

enLista(), fecha(), rango(), hora()

regla($campo, $cumple, $mensaje)

hayErrores(), getErrores()

Cita

La entidad principal

`8 propiedades privadas`

__construct($codigo, $expediente, …)

setMotivo(): máximo 300 caracteres

estaActiva()

cancelar(): solo si está activa

Horario

La segunda entidad

`private $duracion, $dias…`

setDuracion(): 10 a 120, múltiplo de 10

setVigenteDesde(): 01/11/2026 a 02/04/2027

agregarDia($dia, $desde, $hasta, …)

atiende($dia)

Usuario

Cuenta con rol

`private $claveHash`

setClave(): mínimo 8 y password_hash()

verificarClave(): password_verify()

esAdmin()

sin getter para el hash

Idea clave: el constructor usa los setters, y los setters validan; un objeto nunca queda con datos inválidos.

REGLAS DE NEGOCIO

Se validan en el servidor, no solo en el navegador

6 dígitos	**Campo especial**	Número de expediente: preg_match('/^\d{6}$/')
24 h	**Anticipación**	No se reserva con menos de 24 horas
R1	**Sin choque**	Un médico no tiene dos citas activas a la misma hora
R2	**Cupo diario**	Máximo 8 citas activas por médico y día (MAX_CITAS_DIA)
R3	**Una por especialidad**	El paciente no tiene dos citas activas de la misma especialidad
Agenda	**Calendario real**	Feriados, ausencias y horario del médico

ASÍ SE ESCRIBEN EN procesar_principal.php

```
// R1: ¿el médico ya tiene esa hora?
$validador->regla('hora',
    !$hora_ocupada,
    'El médico ya tiene una cita a esa hora.');

// R2: cupo del día
$validador->regla('fecha',
    $activas_dia < MAX_CITAS_DIA,
    'El médico no tiene más cupo ese día.');
```

¿Por qué en el servidor? El atributo required o el pattern del HTML se quitan con las herramientas del navegador; la validación en PHP no.

Cada medida protege contra un riesgo concreto

`htmlspecialchars()`

en limpiar(): el texto escrito se muestra como texto, no como código (XSS)

`password_hash()`

la contraseña nunca se guarda tal cual; password_verify() la compara

`Listas blancas`

isset() e in_array(…, true): un select manipulado se rechaza

`exigir_rol()`

sin sesión → login.php; rol no permitido → error 403

`display_errors = 0`

el usuario no ve rutas ni nombres de archivos

QUÉ VE CADA ROL EN EL MENÚ

Opción	Invitado	Paciente	Recepción	Médico	Admin
Reservar cita	✓	✓	✓	—	—
Citas	—	✓	✓	✓	✓
Horarios	—	—	✓	✓	✓
Administración	—	—	—	—	✓

Ocultar un enlace no protege una página. Por eso cada página llama a exigir_rol(): si un paciente escribe panel.php en la URL, recibe el error 403.

DATOS DE DEMOSTRACIÓN

Los listados salen de arreglos de PHP, no de filas a mano

config/datos_demo.php

5 médicos	**15** pacientes	**17** citas
4 usuarios, uno por rol	**13** feriados	**8** estados de cita

REGISTROS SEMILLA DICTADOS EN CLASE

482915	**Lucía Montenegro-Ríos**	Pediatría	Confirmada
482916	**Teresa D'Alessio**	Dermatología	Pendiente
482917	**Hernán Quijada**	Pediatría	Cancelada

PAGINACIÓN: 5 CITAS POR PÁGINA CON $_GET

```
$por_pagina = 5;
$total_paginas = max(1, (int) ceil(
    $total_filtradas / $por_pagina));
$pagina = (int) ($_GET['pagina'] ?? 1);
$pagina = max(1, min($pagina, $total_paginas));
$citas_pagina = array_slice($citas_filtradas,
    ($pagina - 1) * $por_pagina, $por_pagina);
```

`?pagina=-3`	muestra la página 1
`?pagina=99`	muestra la última
`?pagina=hola`	(int) da 0 → página 1

Cada requerimiento se une a su código y a su prueba

PLAN DE PRUEBAS · 19 CASOS

E **Edwin P-01 a P-06**

XSS, archivos protegidos, contraseñas, formularios

D **Dana P-07 a P-12 y P-19**

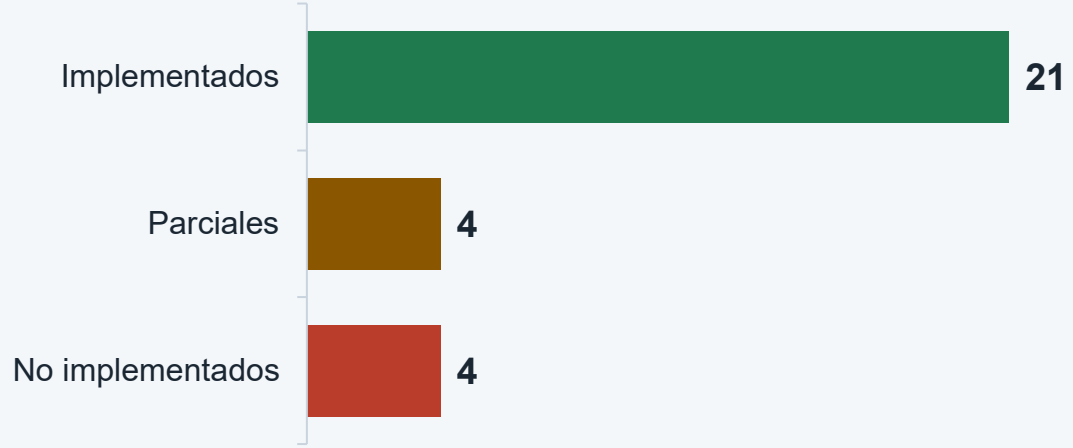
login, acceso por rol, R1, R2, R3 y reserva

A **Alvaro P-13 a P-18**

required quitado, selects, horario, nota, feriados

Cada caso indica los pasos, los datos de entrada, el resultado esperado y la evidencia (captura).

MATRIZ DE TRAZABILIDAD · 29 REQUERIMIENTOS



Lo que falta necesita guardar datos: reprogramar, recuperar la contraseña, administración y reportes llegan con la base de datos del Hito 3.

DEMOSTRACIÓN EN VIVO

Recorrido de unos 3 minutos con los cuatro roles

1

Invitado

Inicio y buscador por GET; el menú muestra solo lo público.

`index.php`

2

Paciente

Reserva con expediente de 3 dígitos → errores y valores conservados. Luego R3 con cardiología.

`principal_formulario.php`

3

Recepción

Listado con filtros y paginación; probar ?pagina=99. Horario de un médico.

`principal_listado.php`

4

Médico

«Mi agenda» solo con sus citas; nota de consulta y «marcar atendida».

`principal_detalle.php`

5

Seguridad

Abrir includes/funciones.php → «Acceso denegado». Paciente en panel.php → 403.

`exigir_rol()`

Antes de empezar: servidor encendido (php -S localhost:8000), sesión cerrada y las cuentas de demostración del README a mano.

CIERRE

Lo que aprendimos y lo que sigue

Aprendimos

Nunca confiar en lo que llega del navegador: validar en el servidor

Separar datos, lógica y presentación (config/, clases/, includes/)

Trabajar con ramas y Pull Requests, y resolver conflictos de Git

Hito 3

Base de datos: las citas y horarios se guardan de verdad

Reprogramar y cancelar, recuperar la contraseña

Administración y reportes de citas y ausentismo

¡Gracias! ¿Preguntas?



El uso de inteligencia artificial está declarado en el README: herramienta, para qué y en qué parte.